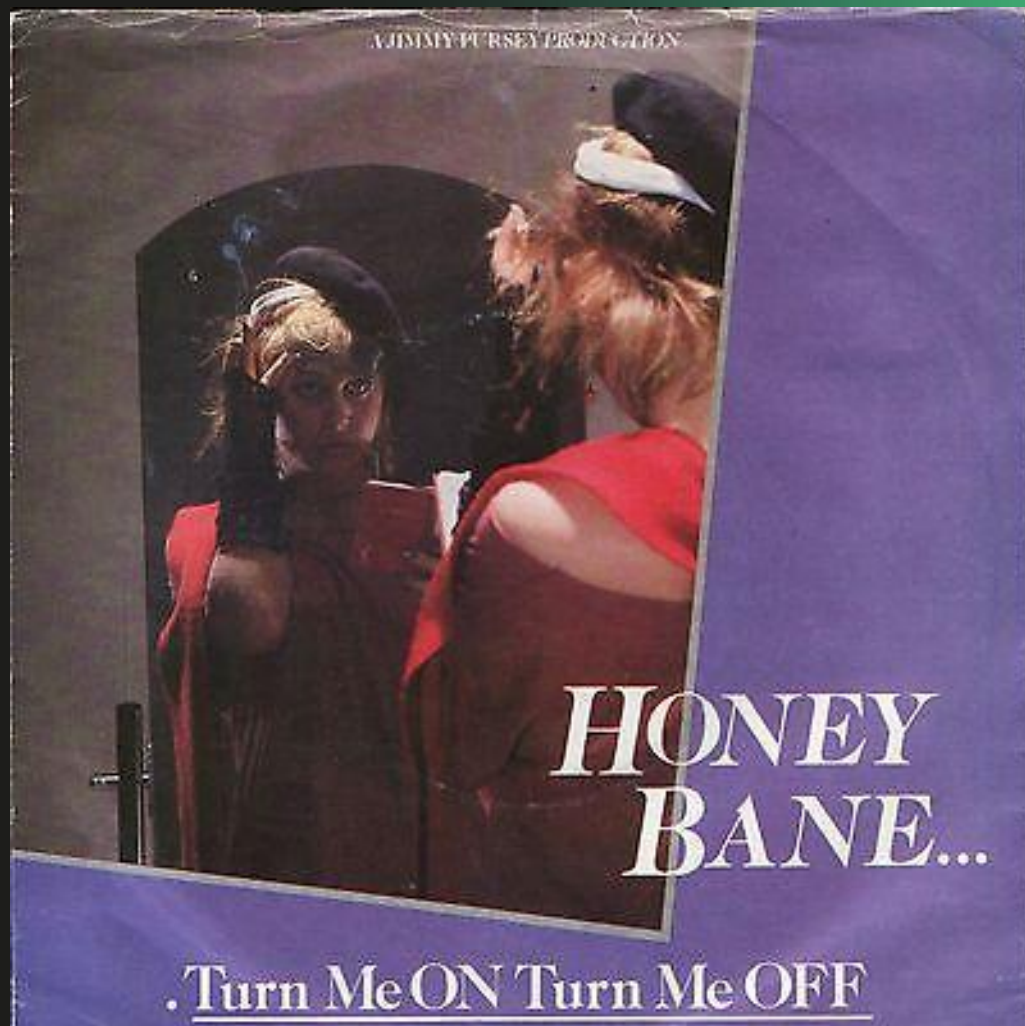




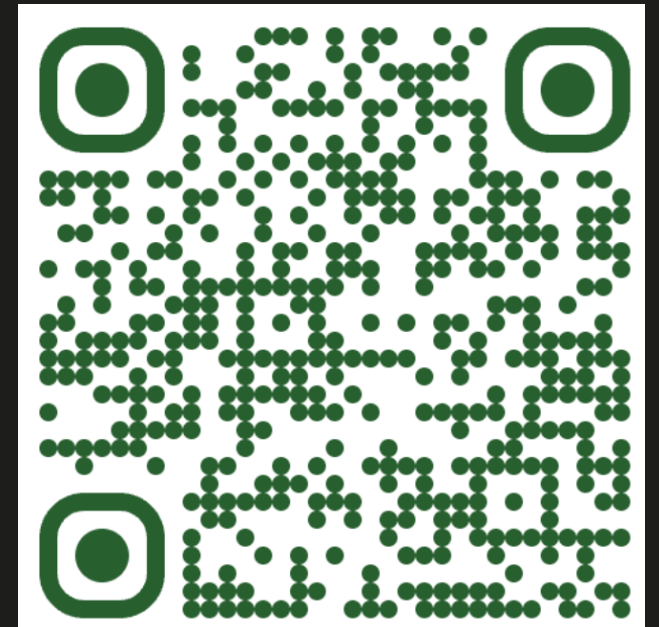
Haidar Kabibo

Kaspersky

- Member of Industrial Security Services team, Kaspersky
- Do Mainly windows researches
- Masters of None:
 - Pentest
 - RE
 - AppSec
 - ICS
 - Network
 - Radio

```
$ echo d2hvYW1pCg== | base64 -d | bash
```

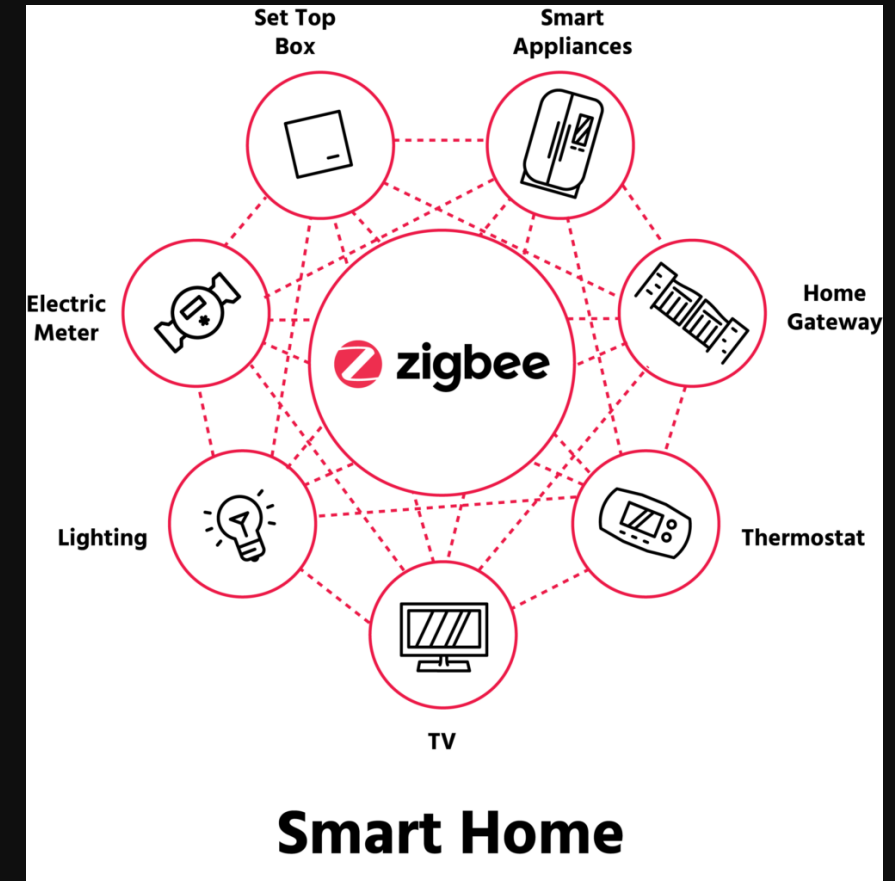
Sud0Ru



What this talk about?

Zigbee

- **Low-power**, wireless communication protocol, WSN
- Based on the IEEE 802.15.4 standard
 - Short-range, low-data-rate applications
 - Mesh networking capability
- Operates in the 2.4 GHz frequency band
- Smart homes, Industrial automation



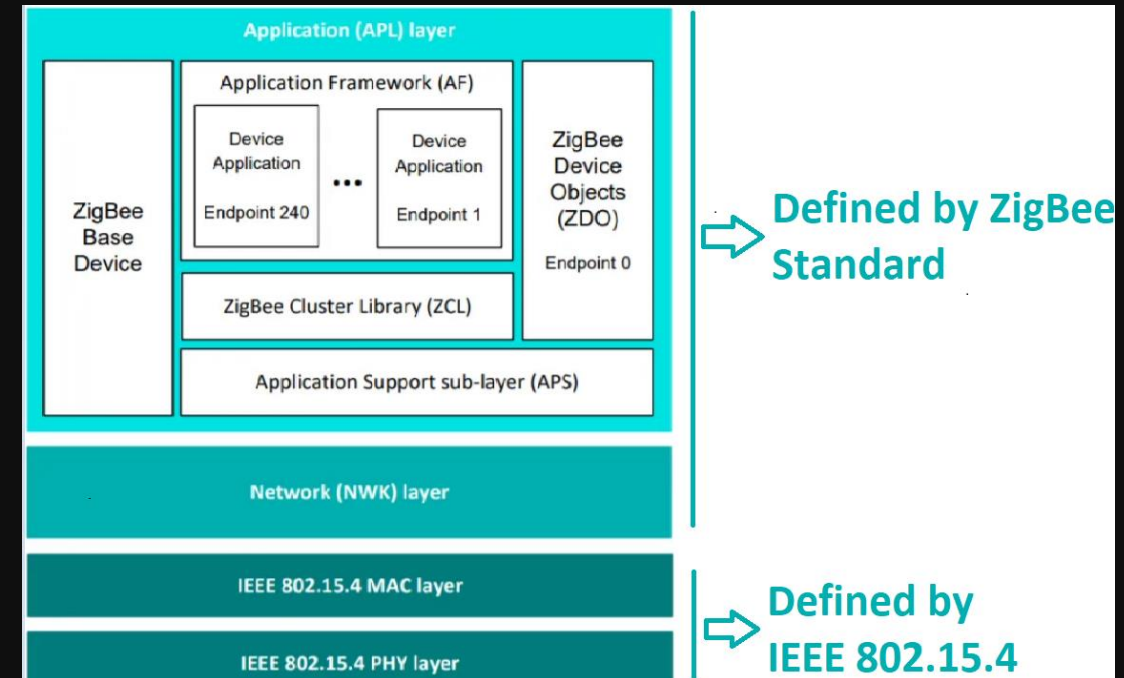
Why Not Wi-Fi?

- Power Consumption
- Network Size and topology
 - Use Case Focus

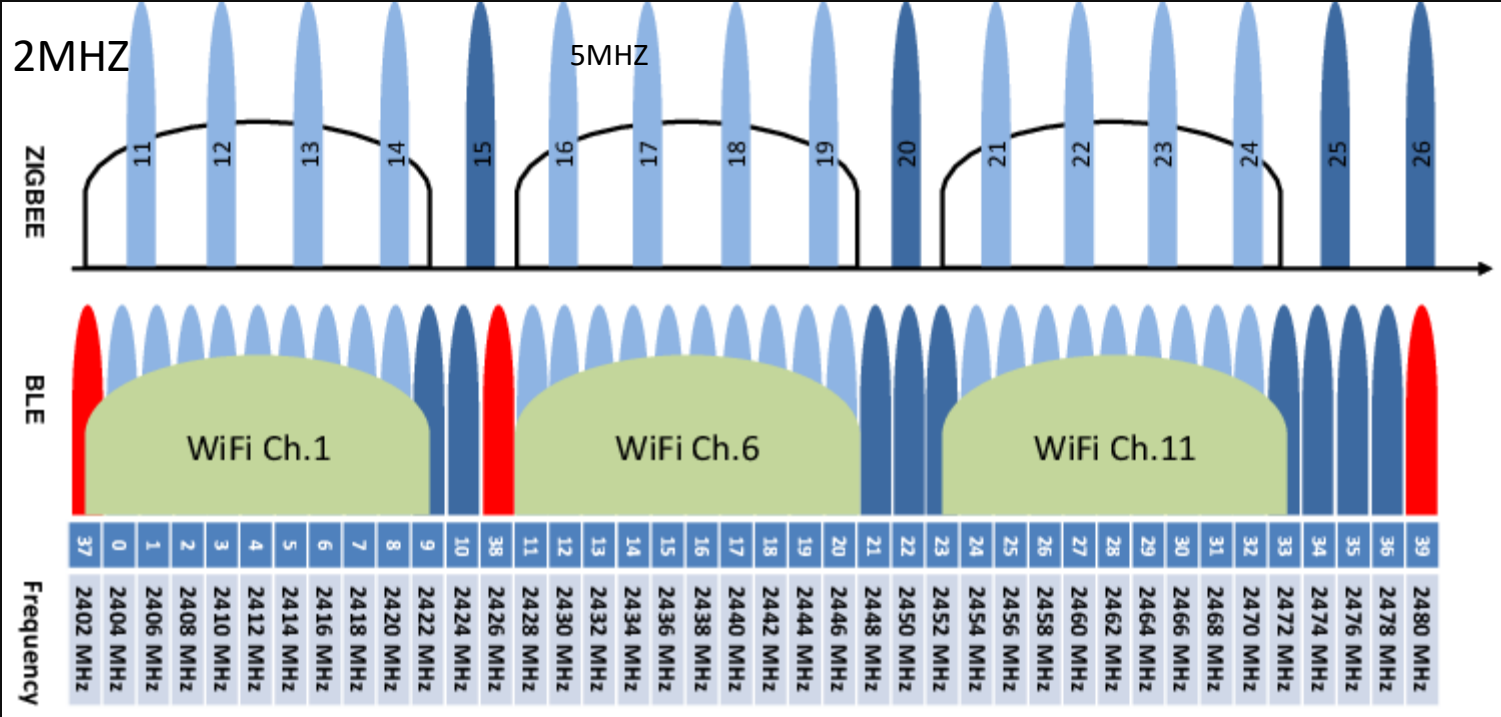


Zigbee Vs 802.15.4

- IEEE 802.15.4 standard is a physical (PHY) and media access control (MAC) layer
- Thread, WirelessHART, 6LoWPAN, MiWi



Zigbee Wireless Band



Frequency Band (MHz)	Frequency Range (MHz)	Modulation Technique	Bit Rate (Kbps)	Country	Number of Channels
868	868-868.6	BPSK	20	Europe	1
915	902-928	BPSK	40	USA and Australia	1-10, 13 (North America)
2400	2400-2483.5	OQPSK	250	Worldwide	16

Zigbee Devices

- Zigbee Coordinator (ZC):

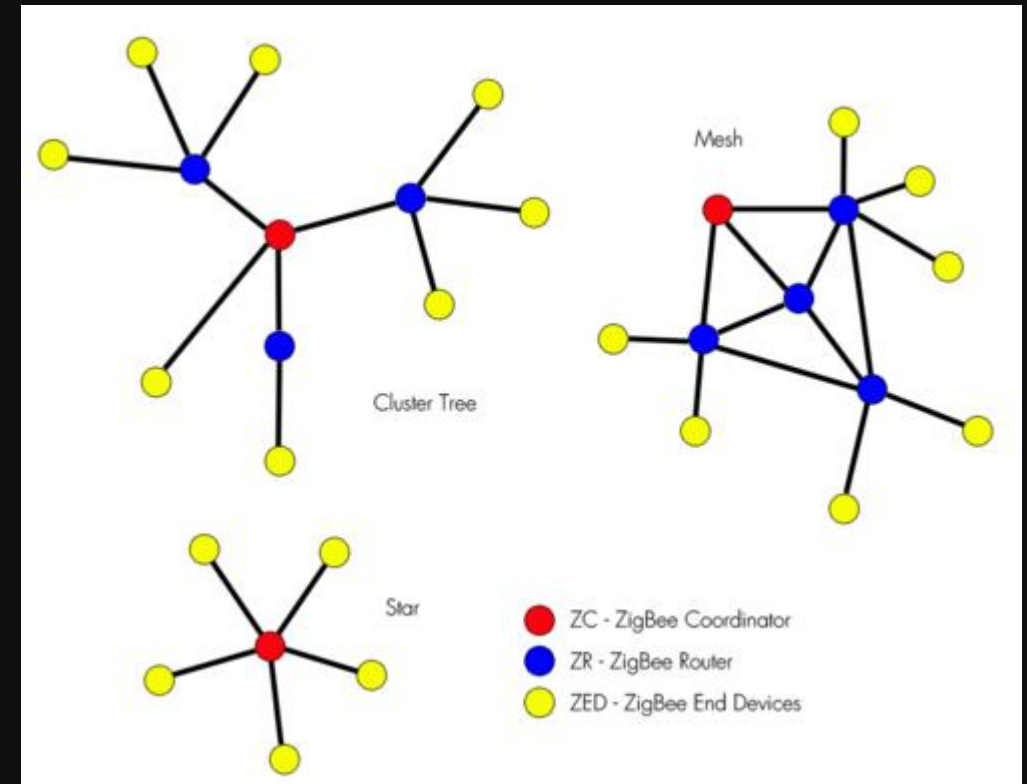
- The brain of the network — there can be only one per Zigbee network.
 - Starts and manages the Zigbee network
- Assigns addresses to other devices.
 - Stores network information.

- Zigbee Router (ZR):

- Forwards data and extends network range (mesh networking).
- Forms the **backbone** of the Zigbee mesh.

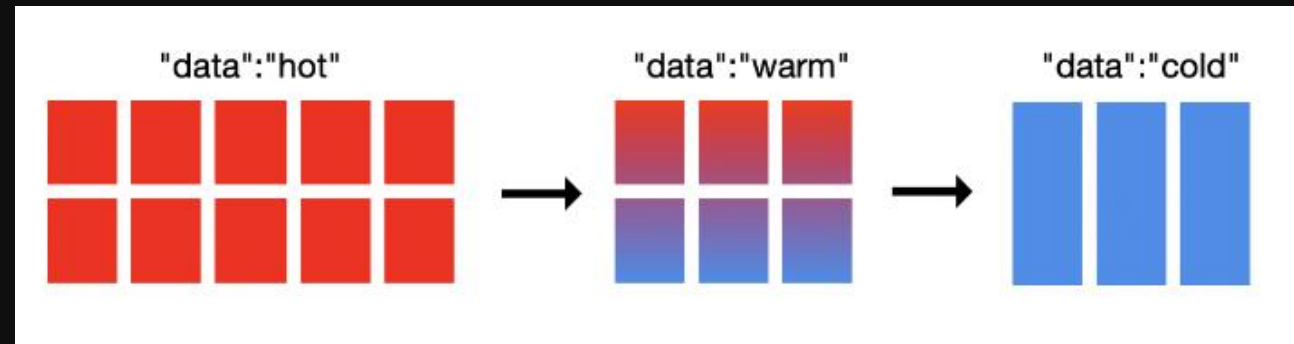
- Zigbee End Device (ZED)

- The simplest and lowest-power role
- Communicates only with its parent.
- Sleeps most of the time to save power.
- Examples: Sensors, remotes, buttons, etc.



Zigbee Network

- The Zigbee network is started by the Coordinator.
 - The Coordinator always has the address 0x0000.
- The Zigbee Coordinator selects the channel for communication.
- It also chooses the PAN ID, which is a 2-byte value (e.g., 0x1234).
- Additionally, the Coordinator sets the Extended PAN ID, which is 8 bytes long.
- This is often an ASCII representation of the network name.
 - For example **HOTNODEZ** 🐼



Zigbee Addressing

- The nodes has:
 - 2 bytes short address
- Extend Address which is MAC address with 8 bytes
- The both addresses cab be used in two layers the MAC and network

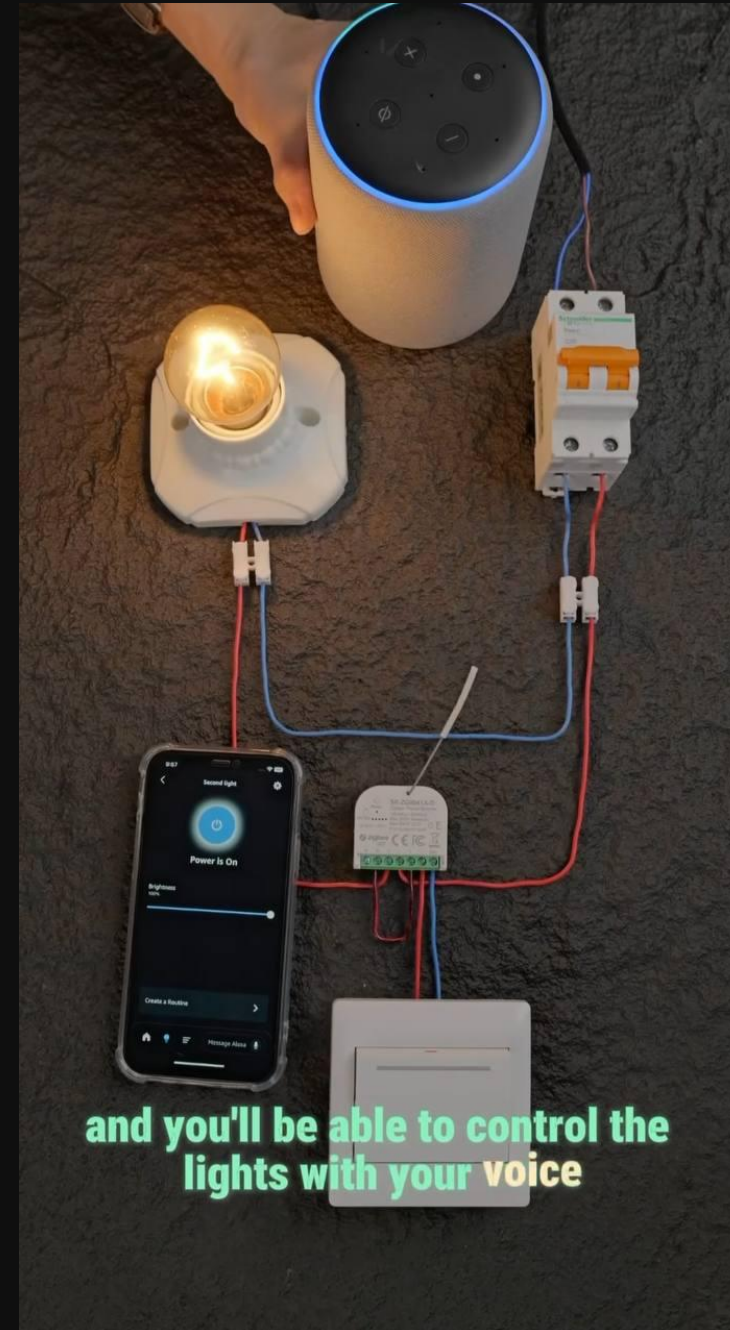
```

▼ IEEE 802.15.4 Data, Src: 0x896b, Dst: Broadcast
  ▶ Frame Control Field: 0x8841, Frame Type: Data, PAN ID Compression, De
    Sequence Number: 176
    Destination PAN: 0xda88
    Destination: 0xffff
    Source: 0x896b
    [Extended Source: NordicSemico_8f:c4:80:3d:c0 (f4:ce:36:8f:c4:80:3d:c0
    [Origin: 31]
▼ ZigBee Network Layer Data, Dst: Broadcast, Src: 0x896b
  ▶ Frame Control Field: 0x1208, Frame Type: Data, Discover Route: Suppre
    Destination: 0xffffd
    Source: 0x896b
    Radius: 30
    Sequence Number: 0
    Extended Source: NordicSemico_8f:c4:80:3d:c0 (f4:ce:36:8f:c4:80:3d:c0

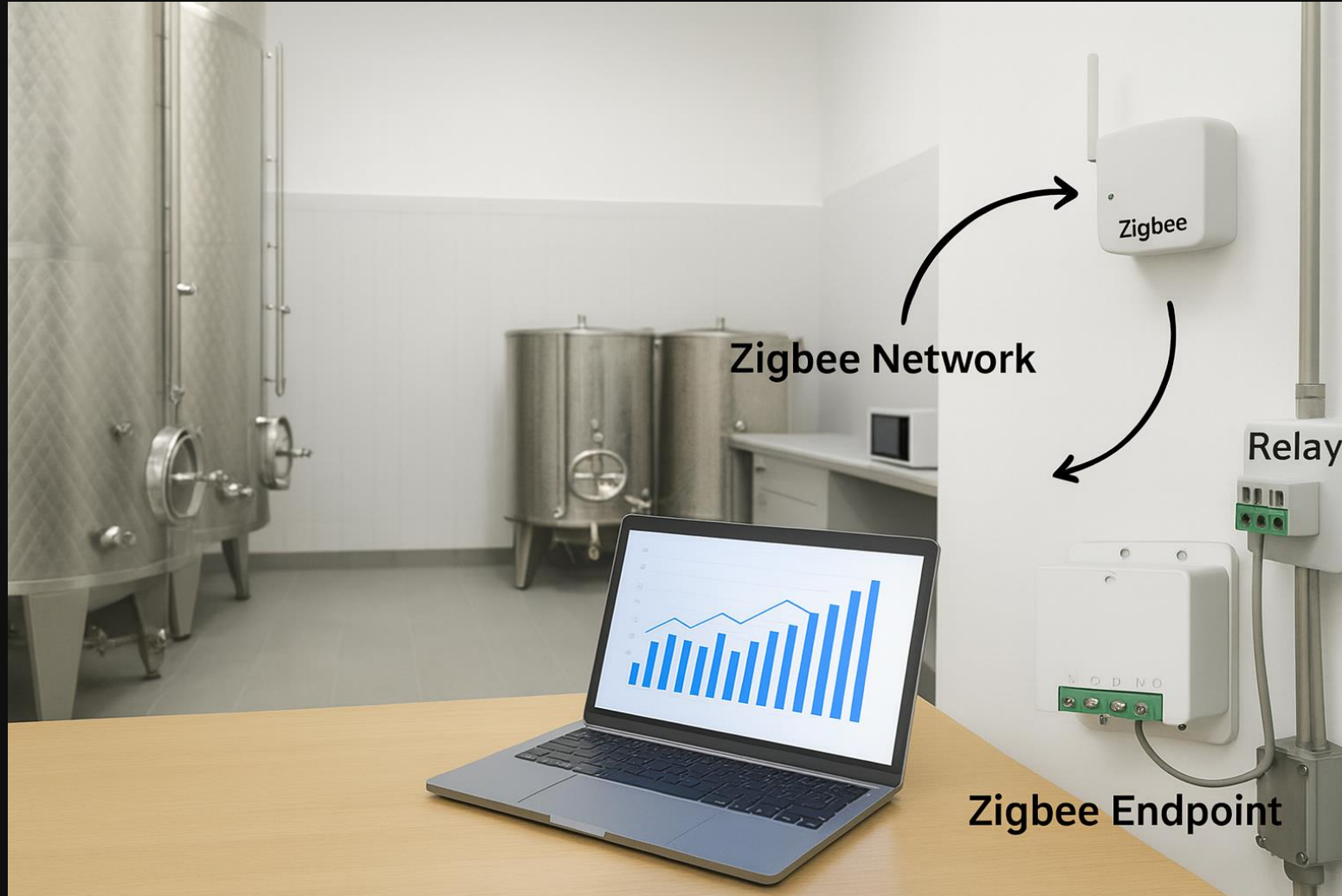
```

Part one:

- The Zigbee network consists of a Coordinator and an End Device, which is connected to a relay.
- The Coordinator has multiple interfaces, including Wi-Fi, Zigbee, LTE, and Bluetooth.
- The Coordinator turns the relay ON and OFF via Zigbee communication.
- The goal was to hijack the connection between the Coordinator and the End Device in order to gain control of the relay and toggle its state.



Industrial setup:



1.1 Sniffing:

- nRF52840 Dongle
 - Monitoring and sniffing of Zigbee communication traffic in real time.
 - It has integration with Wireshark
 - You must know the specific Zigbee channel in use



1.1 Sniffing:

- After identifying the Zigbee channel, you can begin sniffing the traffic.
- The application layer data is encrypted and not visible.
- Only the first two layers — Network and MAC — can be analyzed.
- From this, you can extract details like the PAN ID, source and destination addresses, and sometimes the source and destination MAC addresses.

Time	Source	Destination	Info
64 145.514	0x0000	0xffff	Data, Dst: Broadcast, Src: 0x0000
65 146.005	0x0000	0xffff	Data, Dst: Broadcast, Src: 0x0000
66 154.529	0x0000	0xffff	Command, Dst: Broadcast, Src: 0x0000
67 167.711	0x0000	0xffff	Command, Dst: Broadcast, Src: 0x0000
68 167.995	0x0000	0xffff	Command, Dst: Broadcast, Src: 0x0000
69 168.271	0x0000	0xffff	Command, Dst: Broadcast, Src: 0x0000
70 168.610	0x0000	0xffff	Command, Dst: Broadcast, Src: 0x0000
71 169.665	0x0000	0xffff	Data, Dst: Broadcast, Src: 0x0000
72 170.098	0x0000	0xffff	Data, Dst: Broadcast, Src: 0x0000
73 170.586	0x0000	0xffff	Data, Dst: Broadcast, Src: 0x0000
74 171.640	0x0000	0xffff	Command, Dst: Broadcast, Src: 0x0000
75 175.142	0x0000	0xffff	Command, Dst: Broadcast, Src: 0x0000
76 179.340	0x0000	0xffff	Command, Dst: Broadcast, Src: 0x0000
77 183.109	0x0000	0xffff	Command, Dst: Broadcast, Src: 0x0000
78 199.959	0x0000	0xffff	Command, Dst: Broadcast, Src: 0x0000

```

▼ IEEE 802.15.4 Data, Src: 0x0000, Dst: Broadcast
  ▶ Frame Control Field: 0x8841, Frame Type: Data, PAN ID Compression, D
    Sequence Number: 255
    Destination PAN: 0xac5a
    Destination: 0xffff
    Source: 0x0000
  
```

1.2 Decryption:

The Network key

- The traffic is encrypted with network key
- The Network Key is a symmetric encryption key shared by all devices in a Zigbee network. It secures communication at the Network layer.
- Ensuring data confidentiality and integrity within the network.
- It is usually randomly generated and delivered encrypted by the coordinator to the node using a **Transport Key packet (Transport THE key)**, which is itself encrypted with another key called the Link Key.



1.2 Decryption:

The Link Key

- Secret key shared between a Zigbee device and the coordinator.
- It is used for authentication during the secure joining process.
- It used to **encrypt the Network Key** when it's sent to a new device.
- It can be pre-installed on devices (known ahead of time), or
 - Generated and exchanged using key establishment protocols like SKKE (Symmetric-Key Key Establishment)
- Zigbee Alliance Link Key “**ZigbeeAlliance09**”

```
29 294.0... 0x0000 0x896b Transport Key
30 294.0... Ack

> Frame 29: 87 bytes on wire (696 bits), 87 bytes captured (696 bits) on interface COM22, id 0
> IEEE 802.15.4 Data, Src: 0x0000, Dst: 0x896b
> ZigBee Network Layer Data, Dst: 0x896b, Src: 0x0000
> ZigBee Application Support Layer Command
  > Frame Control Field: Command (0x21)
    Counter: 59
  > ZigBee Security Header
  > Command Frame: Transport Key
    Command Identifier: Transport Key (0x05)
    Key Type: Standard Network Key (0x01)
    Key: 752a28d2b537c366726e34dd32cfcf85
    Sequence Number: 0
    Extended Destination: NordicSemico_8f:c4:80:3d:c0 (f4:ce:36:8f:c4:80:3d:c0)
    Extended Source: NordicSemico_df:8d:12:5d:23 (f4:ce:36:df:8d:12:5d:23)
```

1.2 Decryption:

The choice

1- Capture the Network Key During Device Joining through:

- This requires access to the Link Key.
- In most cases, the Link Key is the default Zigbee Alliance key (ZigbeeAlliance09).
- You must wait for a device to leave and rejoin the network — during rejoining, the Network Key is sent, encrypted with the Link Key.
- If you capture this rejoin process, you can decrypt the Network Key using the known Link Key

2- Guessing the Network key which is most likely randomly generated

1.2 Decryption:

Obtaining the keys

1. Try Default Keys First

- Start by testing well-known default keys like the Zigbee Alliance key (ZigbeeAlliance09).
 - Many commercial devices use this key for initial secure joining.

2. Identify the Manufacturer

- If the default key doesn't work, try to identify the manufacturer of the Zigbee chip.
 - You can do this in two ways:
- A. Check the MAC address of the Zigbee device — the first few bytes usually indicate the manufacturer.
 - B. Or physically inspect the device for hardware information (model number, chip label, etc.).

3. Look Up Manufacturer-Specific Keys

- Some manufacturers use their own default Link Keys.
- Once the manufacturer is known, search for known default keys used by that brand.
 - 4. Extract the firmware
- If the previous stage didn't work you can extract the firmware from the chip and search for the keys

1.2 Decryption:

Obtaining the keys

- Extract the firmware from the Zigbee chip to obtain the secret keys directly.
- Search public sources for known default keys used by our chip manufacture
- In our case, we successfully obtained the keys using both methods.

1.2 Decryption:

Final Success

- At this stage of the assessment, we are only interested in the Network Key.
- Once added to Wireshark, we were able to successfully decrypt the Zigbee traffic

Time	Source	Destination	Info
42 78.333	0x0000	0xffff	ZCL: Read Attributes, Seq: 22[Malformed Packet]
43 78.778	0x0000	0xffff	ZCL: Read Attributes, Seq: 22[Malformed Packet]
44 79.719	0x0000	0xffff	Link Status
45 79.842	0x0000	0xffff	Link Status
46 84.181	0x0000	0xffff	Link Status
47 84.924	0x0000	0xffff	ZCL: Read Attributes, Seq: 22[Malformed Packet]
48 85.411	0x0000	0xffff	ZCL: Read Attributes, Seq: 22[Malformed Packet]
49 85.844	0x0000	0xffff	ZCL: Read Attributes, Seq: 22[Malformed Packet]
50 88.081	0x0000	0xffff	Link Status
51 91.882	0x0000	0xffff	Link Status
52 106.499	0x0000	0xffff	Link Status
53 114.979	0x0000	0xffff	ZCL: Read Attributes, Seq: 22[Malformed Packet]

1.3 TurnMeOff TurnMeOn:

Choose your hardware

- After decrypting the traffic and understanding the communication between the coordinator and the endpoint, the next step is to select hardware for injecting Zigbee packets into the network.
- Common and Accessible Options in our case:
 - 1- CC2531 USB Dongle – simple and widely supported.
 - 2- SDR (Software Defined Radio) such as BladeRF – offers more flexibility and control.



1.3 TurnMeOff TurnMeOn: CC2531

- We faced some issues using BladeRF with the Zigbee stack implemented in GNU Radio.
- As a result, we relied on the CC2531 USB dongle as our primary solution.
- It is compatible with the KillerBee framework, which is designed for Zigbee auditing.
- There is custom firmware available for the CC2531 that enables packet injection through the USB interface using killerbee.
- This firmware can be uploaded using a Raspberry Pi, as an alternative to the CC debugger.



1.3 TurnMeOff TurnMeOn:

Crafting the packet

- When analyzing Zigbee communication in Wireshark, you may notice many malformed packets.
- This usually means that Wireshark couldn't fully interpret the packets due to certain limitations or missing information.
- To understand why this happens, we need to take a closer look at the Zigbee Application Layer.

Time	Source	Destination	Info
42 78.333	0x0000	0xffff	ZCL: Read Attributes, Seq: 22[Malformed Packet]
43 78.778	0x0000	0xffff	ZCL: Read Attributes, Seq: 22[Malformed Packet]
44 79.719	0x0000	0xffff	Link Status
45 79.842	0x0000	0xffff	Link Status
46 84.181	0x0000	0xffff	Link Status
47 84.924	0x0000	0xffff	ZCL: Read Attributes, Seq: 22[Malformed Packet]
48 85.411	0x0000	0xffff	ZCL: Read Attributes, Seq: 22[Malformed Packet]
49 85.844	0x0000	0xffff	ZCL: Read Attributes, Seq: 22[Malformed Packet]
50 88.081	0x0000	0xffff	Link Status
51 91.882	0x0000	0xffff	Link Status
52 106.499	0x0000	0xffff	Link Status
53 114.979	0x0000	0xffff	ZCL: Read Attributes, Seq: 22[Malformed Packet]

1.3 TurnMeOff TurnMeOn:

Crafting the packet: Application layer

Zigbee Application Profiles

- The profile is an agreement between applications on different devices for message handling
- Defines logical components and their interfaces
 - Enables Interoperability across different manufacturers

Types of Zigbee Profiles

- Public (Standard) – Managed by the Connectivity Standards Alliance (CSA)
- Private – Defined by Zigbee vendors for restricted use
- Published – Previously private profiles that are later made public

Endpoints

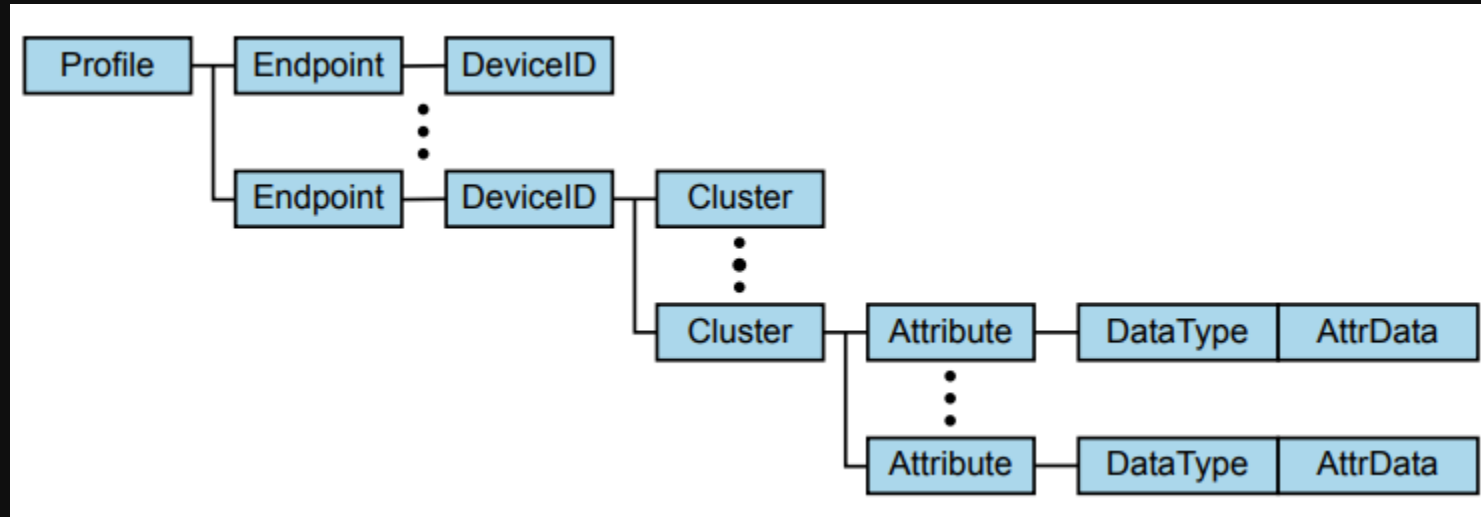
- An endpoint is a logical interface on a Zigbee device
- Each endpoint represents a specific application or function (e.g., light control, temperature sensor)
- A single Zigbee device can host multiple endpoints to support different functionalities
- There is some special endpoints number like 0 or 255
 - 0: Reserved for Zigbee Device Object (ZDO)
 - 255: Used for broadcasts to all endpoints

Clusters

- Define a device's capabilities (commands and attributes)
- Managed by the Zigbee Cluster Library (ZCL)
 - Each cluster has a client and server side

1.3 TurnMeOff TurnMeOn:

Crafting the packet: Application layer



▼ ZigBee Application Support Layer Data, Dst Endpt: 1, Src Endpt: 1

- ▶ Frame Control Field: Data (0x40)
 - Destination Endpoint: 1
 - Cluster: On/Off (0x0006)
 - Profile: Home Automation (0x0104)
 - Source Endpoint: 1
 - Counter: 231

▼ ZigBee Cluster Library Frame

- ▶ Frame Control Field: Cluster-specific (0x11)
 - Sequence Number: 161
 - Command: Off (0x00)

▼ ZigBee Application Support Layer Data, Dst Endpt: 0, Src Endpt: 0

- ▶ Frame Control Field: Data (0x00)
 - Destination Endpoint: 0
 - Simple Descriptor Request (Cluster ID: 0x0004)
 - Profile: ZigBee Device Profile (0x0000)
 - Source Endpoint: 0
 - Counter: 220

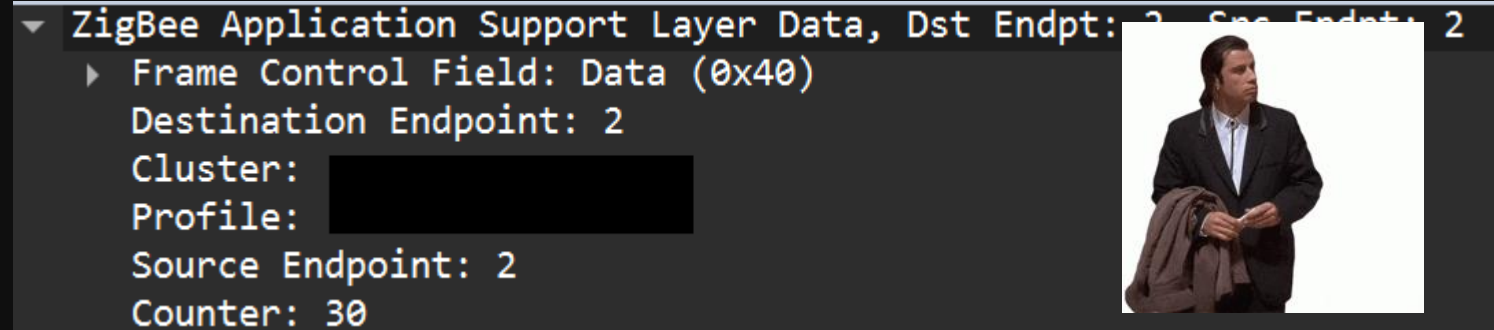
▼ ZigBee Device Profile, Simple Descriptor Request, Nwk Addr: 0x50a4, Endpoint: 1

- Sequence Number: 8
- Nwk Addr of Interest: 0x50a4
- Endpoint: 1

1.3 TurnMeOff TurnMeOn:

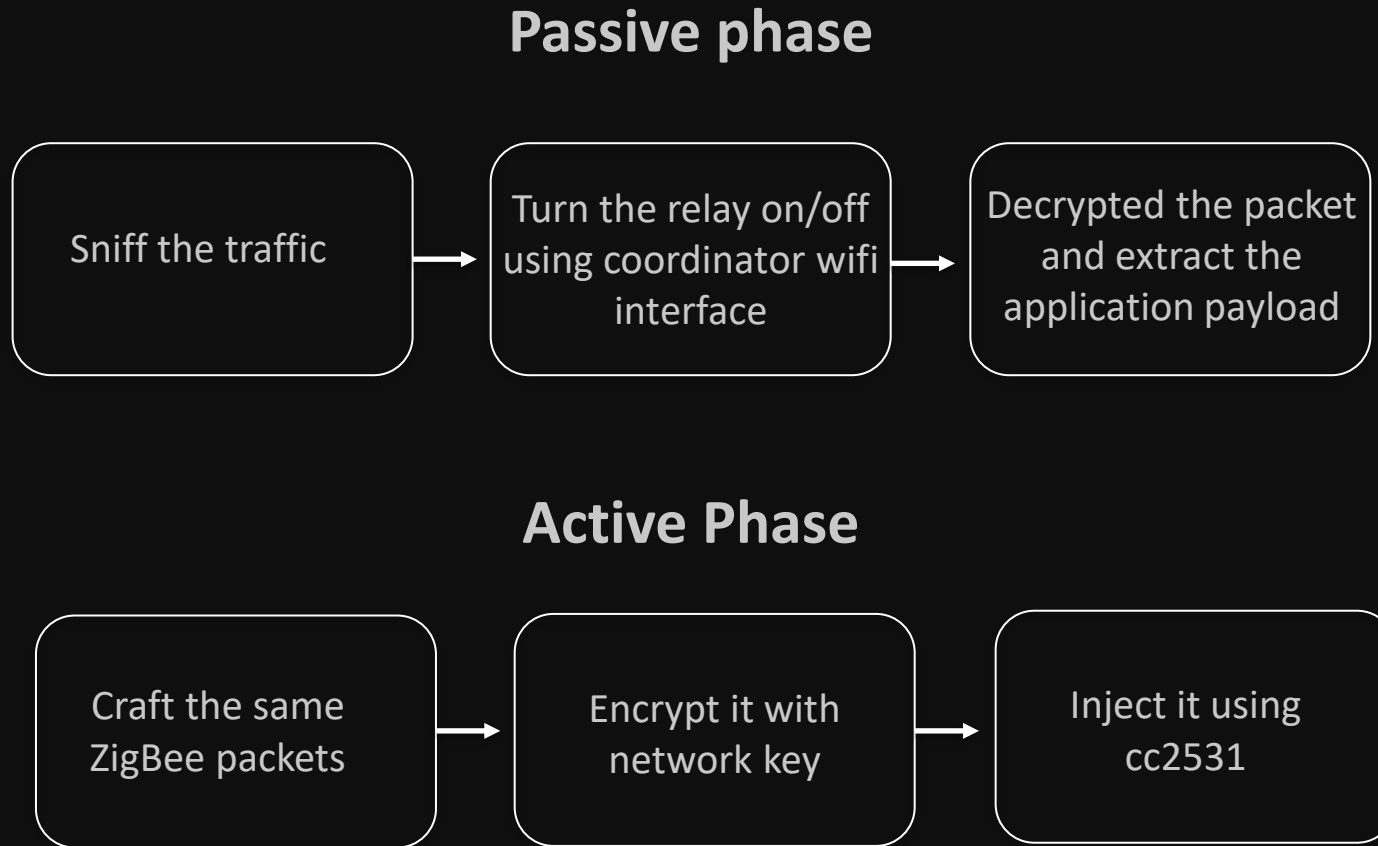
Crafting the packet: Forgot it all

- Our Zigbee network uses a proprietary profile , with Private Profile ID.
- This is why Wireshark couldn't properly interpret the application-layer packets — the profile is not a public profile, and the implementation is left to the developers
- To control the switch (turn ON/OFF), we now need to understand how the endpoints and clusters are structured in this custom profile



1.3 TurnMeOff TurnMeOn:

Crafting the packet: The flowchart



1.3 TurnMeOff TurnMeOn:

Crafting the packet: Scapy

```
from scapy.layers.dot15d4 import Dot15d4, Dot15d4FCS, Dot15d4Data, Dot15d4Cmd, Dot15d4Beacon, Dot15d4CmdAssocResp
from scapy.layers.zigbee import ZigbeeNWK, ZigbeeAppDataPayload, ZigbeeSecurityHeader, ZigBeeBeacon, ZigbeeAppCommandPayload
# from scapy.layers.zigbee import APS
```

```
def craft_pkt():
    global cc
    cc += 1

    aps = ZigbeeAppDataPayload(
        frame_control=APS_ACK_REQUEST,
        delivery_mode=0,
        aps_frame_type=0,

        dst_endpoint=,
        cluster=,
        profile=0x0000,
        src_endpoint=,

        counter= cc % 256
    )

    plaintext_payload = aps.__bytes__() + relay_off

    sec = ZigbeeSecurityHeader(
        extended_nonce=1,
        key_type=1,
        nwk_seclevel=5, # ENC-MIC-32
```

```
def encrypt_and_sign_packet(nwk_header: ZigbeeNWK, sec_header: ZigbeeSecurityHeader):
    nonce = craft_nonce(sec_header)

    cipher = AES.new(NWK_KEY, AES.MODE_CCM, nonce, mac_len=4)
    cipher.update(bytes(nwk_header) + bytes(sec_header))
    ciphertext = cipher.encrypt(plaintext_payload)

    sec_header.data = ciphertext
    sec_header.mic = cipher.digest()
    sec_header.nwk_seclevel = 0

    return nwk_header / sec_header
```

```
def craft_nonce(sec_layer: ZigbeeSecurityHeader) -> bytes:
    sec_control_field = bytes(sec_layer)[0].to_bytes(1, 'little')
    nonce = [
        sec_layer.source.to_bytes(8, 'little') +
        sec_layer.fc.to_bytes(4, 'little') +
        sec_control_field
    ]

    return nonce
```

```
kbsendp(pkt, channel=18, inter=0, loop=0, iface=None, verbose=None, realtime=None)
```

1.3 TurnMeOff TurnMeOn:

Crafting the packet: Reply attack

- Our current strategy won't work for one reason.
- To prevent replay attacks, the frame counter in the security header must be increased with each new packet received by the receiver
Also there are other sequence numbers on the applicant level.
- In this case, we need to extend our strategy.

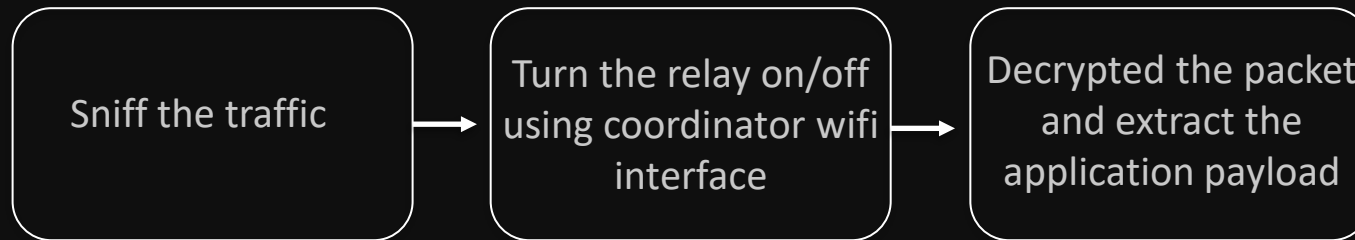
ZigBee Security Header

```
▸ Security Control Field: 0x28, Key Id: Network Key, Extended Nonce  
Frame Counter: 1028  
Extended Source: NordicSemico_df:8d:12:5d:23 (f4:ce:36:df:8d:12:5d:23)  
Key Sequence Number: 0  
Message Integrity Code: d02dee88  
[Key: 752a28d2b537c366726e34dd32cfcf85]
```

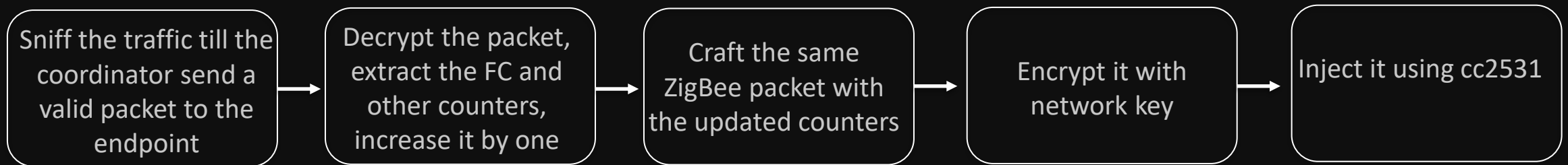
1.3 TurnMeOff TurnMeOn:

Crafting the packet: The flowchart V2

Passive phase



Active Phase



1.3 TurnMeOff TurnMeOn:

Crafting the packet: The flowchart V3

- Working with the CC2531 is not efficient.
- The dongle can't receive and transmit at the same time.
- We switched to using the nRF52840 dongle for both transmission and reception.
- By adding some modifications to the Nordic Sniffer for 802.15.4 source code, we were able to create our own transmitter.
- The Zephyr OS provides a radio API that allows not only receiving but also sending raw Zigbee packets.
- Now using serial communication we can send the same scapy python packets but using the nrf52480 dongle through usb

```
#include <zephyr/net/ieee802154_radio.h>
```

```
static struct ieee802154_radio_api *radio_api;
```

```
int ret = radio_api->tx(radio_dev, tx_mode, pkt, frag);
```

```
serial.Serial('/dev/ttyACM1', 115200, timeout=0.1)
```

1.4 SpoofMe:

Force leaving

- The goal here was to force the Zigbee endpoint to leave its original coordinator's network and connect to our spoofed network, allowing us to take full control of the device.
- To achieve this, we needed to accomplish two main objectives:
 - 1. Force the endpoint to leave the old network
- 2. Spoof the old coordinator and trick the node into connecting to our coordinator instead
 - To force the node to leave, we leveraged a Zigbee protection mechanism that governs the interference between two Zigbee PANs.
 - If a coordinator receives beacons from another nearby coordinator using the same PAN ID, it triggers a PAN ID conflict resolution.
 - In response, the coordinator sends a PAN ID update command to its nodes, instructing them to switch to a new PAN ID.
 - As a result, all nodes must leave the network and rejoin, creating an opportunity to redirect them to our spoofed coordinator

7	41.991	0x0000		Beacon, Src: 0x0000, EPID: :71:74
8	41.996	0x0000	0xffff	Network Update, PAN Identifier Update
9	41.998	0xe8fa	0x0000	Network Report, PAN Identifier Conflict
10	42.000			Ack
11	42.053	0x0000	0xffff	Network Update, PAN Identifier Update
12	42.438	0x0000	0xffff	Network Update, PAN Identifier Update
13	42.924	0x0000	0xffff	Network Update, PAN Identifier Update
14	43.052	0xe8fa	0xffff	Leave
15	43.519	0xe8fa	0xffff	Leave
16	43.971	0xe8fa	0xffff	Leave

```
def craft_beacon():
    p = Dot15d4(
        fcf_frametype=0,
        fcf_srcaddrmode=2,
        fcf_destaddrmode=0,
        fcf_panidcompress=0,
        fcf_ackreq=0,

        seqnum= 1

    ) / Dot15d4Beacon(
        src_panid=PAN_ID,
        src_addr= 0x0000,
        sf_assocpermit=1,
        sf_pancoord=0,
        gts_spec_permit=0,
        gts_spec_desccount=0
```

1.4 SpoofMe:

Beaconing

- Beacon frames are periodic signals sent by the Zigbee coordinator (and routers in beacon-enabled networks).
 - They announce the presence of the network and provide vital information to nearby devices.
 - Helps new devices discover and join the network
 - Maintains synchronization in beacon-enabled networks
- Beacon Frame Includes: PAN ID, Extend PANID, Coordinator address
Stack profile, Device capacity
- When a device wants to join a Zigbee network, it initiates the process by sending a Beacon Request to all ZigBee channels and then waits for Beacon frames from nearby coordinators or routers.
 - If the network with correct extend PANID it will connect to it

```

IEEE 802.15.4 Beacon, Src: 0x0000
  ▶ Frame Control Field: 0x8000, Frame Type: Beacon, Destination Addressing Mo
    Sequence Number: 126
    Source PAN: 0xac5a
    Source: 0x0000
  ▶ Superframe Specification: PAN Coordinator
    .... 1111 = Beacon Interval: 15
    .... 1111 .... = Superframe Interval: 15
    .... 1111 .... = Final CAP Slot: 15
    ...0 .... = Battery Extension: False
    .1... .... = PAN Coordinator: True
    0... .... = Association Permit: False
  
```

```

ZigBee Beacon, Network Specific, EPID
Protocol ID: 0
  ▶ Beacon: Stack Profile: Network Specific, Router Capacity, End Device Capacity
    .... 0000 = Stack Profile: Network Specific (0x0)
    .... 0010 .... = Protocol Version: 2
    .... .1... .... = Router Capacity: True
    .000 0... .... = Device Depth: 0
    1... .... = End Device Capacity: True
    Extended PAN ID:
    Tx Offset: 16777215
    Update ID: 1
  
```

1.4 SpoofMe:

Choose me

- Update ID: Track changes in the network configuration and Help devices decide which coordinator's beacon is newer or more authoritative
- Every time the coordinator performs a network update (e.g., PAN ID change), it increments the Update ID
- If there are two coordinators with the same extended PAN ID but different (or even the same) PAN IDs, and a node needs to decide which one to join, the behavior becomes undefined.
- When craft a beacon with an Update ID higher than the original coordinator's, the endpoint choose our network instead due to.

```
▼ ZigBee Beacon, Network Specific, EPID
  Protocol ID: 0
  ▼ Beacon: Stack Profile: Network Specific, Router Capacity, End Device Capacity
    .... 0000 = Stack Profile: Network Specific (0x0)
    .... 0010 .... = Protocol Version: 2
    .... .1.. .... = Router Capacity: True
    .000 0... .... = Device Depth: 0
    1... .... .... = End Device Capacity: True
  Extended PAN ID: 
  Tx Offset: 16777215
  Update ID: 1
```

1.4 SpoofMe:

The first package of the problems

- The endpoints did choose our coordinator, but the connection was unstable — sometimes they responded to our beacons, and sometimes they didn't.
- It seems we need to respond very quickly, which is difficult to achieve using Python.
- The second response from our coordinator during the connection initiation (we'll cover this later) must also be very fast.
- The endpoint expects an ACK for every packet during the connection setup; if it doesn't receive an ACK in time, it assumes the connection is unstable and drops it.
- Sending timely ACKs in Python is nearly impossible due to latency limitations

Life is getting better



Part two:

- After some time, the customer wanted to test another type of device with the same functionality.
- This time, there was no coordinator — only the endpoint device.
- The problematic package returned... and this time, it came **spicier**



2.1 Reconnaissance:

- By hopping across all Zigbee channels, we were able to detect traffic coming from the device.
 - Turns out, the device was still using the same network key.
 - That allowed us to decrypt the entire network traffic
 - It looked like the device was still connected to its old coordinator.
- The device was acting as a Zigbee Router (ZR). When we sent a Beacon Request, it politely replied with a Beacon Response — and from that, we were able to extract the old PAN ID and Extended PAN ID.

```
> Frame 17: 26 bytes on wire (208 bits), 26 bytes captured (208 bits) on interface
✓ IEEE 802.15.4 Beacon, Src: 0x4b1d
  > Frame Control Field: 0x8000, Frame Type: Beacon, Destination Addressing Mode:
    Sequence Number: 82
    Source PAN: 0x8262
    Source: 0x4b1d
  > Superframe Specification:
  > GTS
    Pending Addresses: 0 Short and 0 Long
✓ ZigBee Beacon, Network Specific, EPID: 
  Protocol ID: 0
  ✓ Beacon: Stack Profile: Network Specific, Router Capacity, End Device Capacity
    .... 0000 = Stack Profile: Network Specific (0x0)
    .... 0010 .... = Protocol Version: 2
    .... 1... .... = Router Capacity: True
    .000 1... .... = Device Depth: 1
    1... .... .... = End Device Capacity: True
  Extended PAN ID: 
  Tx Offset: 16777215
  Update ID: 0
```

2.1 Make Me leave:

- To spoof a new coordinator, we first need to make the device leave its old network.
- That means crafting a specific Zigbee packet, encrypted using the known network key.
 - But here's the catch: we don't know the frame counter from the old coordinator — and without it, the endpoint is unlikely to accept our spoofed packet.
- After some deep diving, I discovered something interesting in the their Zigbee stack:
- If we craft a Zigbee packet using the same network address as the endpoint, encrypt it with the correct network key and broadcast it,
- but use a different MAC address in the security header — the device gets confused and triggers an Address Conflict

975	10279.7...	0x4b1d	0xffff	185065	255	255	Network Update,	PAN Identifier Update
976	10279.7...				255		Ack	
977	10279.7...	0x1d99	0xffffd	185053	37	164	Network Status,	0x4b1d: Address Conflict
978	10279.7...	0x1d99	0xffffd	185054	38	163	Leave	
979	10280.2...	0x1d99	0xffffd	185055	39	164	Network Status,	0x4b1d: Address Conflict
980	10280.2...	0x1d99	0xffffd	185056	40	163	Leave	
981	10280.7...	0x1d99	0xffffd	185057	41	164	Network Status,	0x4b1d: Address Conflict
982	10280.7...	0x1d99	0xffffd	185058	42	163	Leave	
983	10337.2...				50		Beacon Request	

2.2 SpoofMe Badly:

Back to the problem package

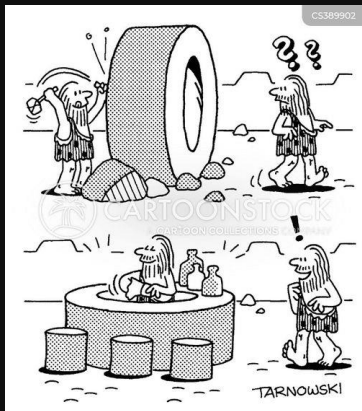
- We returned to the spoofed coordinator issues we faced earlier.
- This time, there was no other coordinator competing with us

The Realization:

We weren't just faking Zigbee — we were basically trying to rebuild parts of Zigbee and 802.15.4, and do it faster and more efficiently than Python would ever allow.

Two Major Problems:

- We needed to respond insanely fast
- We needed to handle Auto ACKs properly



The Pivot:

- So I asked myself: "Why reinvent the wheel?"
- We already had an nrf52048 dongle, which could act as a Zigbee coordinator if we flashed it with the right firmware.
- By setting the correct parameters, we could make the endpoint connect to our spoofed coordinator —and finally, everything would just work

2.2 SpoofMe Badly: Stack profile

- I flushed the nrf52480 dongle with the coordinator's firmware and set all the parameters correctly
 - The timing was great but still I've no response from the endpoint
- After some researching it appears that the problem related to stack profile in Beacon
- Stack profile: This field ensures compatibility between devices attempting to join the network and the network itself
 - 0x00: Network specific ,
 - 0x01: 2004/2006 stack (rarely used today).
 - 0x02: Zigbee Pro (2007+) stack profile (most common).
- It appears that it's impossible to make the nordic dongle work with 0x00: Network-Specific Profile.

```

ZigBee Beacon, Network Specific, EPID: [REDACTED]
Protocol ID: 0
  Beacon: Stack Profile: Network Specific, Router Capacity, End Device Capacity
    .... 0000 = Stack Profile: Network Specific (0x0)
    .... 0010 = Protocol Version: 2
    .... .1.. = Router Capacity: True
    .000 0... = Device Depth: 0
    1... = End Device Capacity: True
Extended PAN ID: [REDACTED]
Tx Offset: 16777215
Update ID: 1
  
```

```

ZigBee Beacon, ZigBee PRO, EPID: NordicSemico_df:8d:12:5d:23
Protocol ID: 0
  Beacon: Stack Profile: ZigBee PRO, Router Capacity, End Device Capacity
    .... 0010 = Stack Profile: ZigBee PRO (0x2)
    .... 0010 = Protocol Version: 2
    .... .1.. = Router Capacity: True
    .000 0... = Device Depth: 0
    1... = End Device Capacity: True
Extended PAN ID: NordicSemico_df:8d:12:5d:23 (f4:ce:36:df:8d:12:5d:23)
Tx Offset: 16777215
Update ID: 0
  
```

2.2 SpoofMe Badly:

Back to the roots

- It seems there was no way around it — I had to **reinvent the wheel**.
- While digging deeper into Zephyr OS, I discovered that you can actually implement an 802.15.4 coordinator (covering the first two layers of the Zigbee stack),
- including auto ACK for every packet that matches the node's PAN ID and network address.
- In our old setup, we used a promiscuous mode node, which received every packet, regardless of destination.
- In the new implementation, we're able to set the PAN ID and network address, so the dongle only accepts packets meant for our node — and just like that, auto ACK is handled!

I should implement it in python

Beaconing, Auto Ack for 802.15.4 implemented using Zyper OS
Zigbee Connection initiation using hardcoded bytes

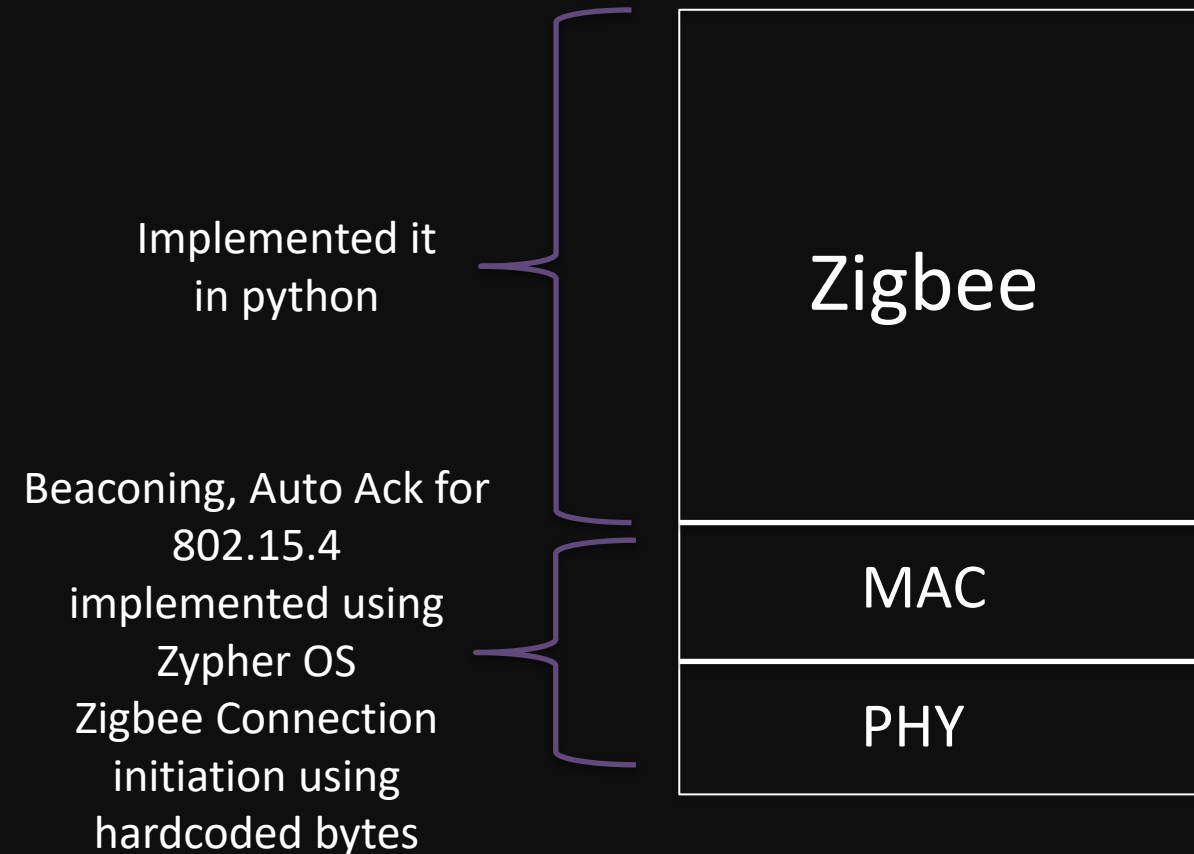
Zigbee

MAC

PHY

2.2 SpoofMe Badly: Back to the roots

- Now, I started building the rest of the Zigbee stack in Python — everything above the first two layers.
- The result? Around 1000 lines of Python code acting as our Zigbee coordinator logic.
- The low-level 802.15.4 Medium Access Control algorithms) + ACK logic is handled by the dongle (C)
 - The Zigbee protocol logic is handled in Python
- During connection initiation, timing is critical — we must respond immediately.
 - But doing that in Python? Too slow.
- So the fix? We pre-generated the necessary bytes in Python and hardcoded them into the dongle firmware.
- With some clever if statements, the dongle could instantly respond, solving the last performance bottleneck.



2.2 SpoofMe Badly:

Connection init

- When a Zigbee endpoint wants to join a network, it broadcasts Beacon Requests across all Zigbee channels.
- It then waits for Beacon Responses from nearby coordinators.
- Based on factors like signal strength and Extended PAN ID, the node decides which coordinator to join.
- If it chooses a coordinator, the node sends an Association Request to that coordinator's address.
- After that, it sends a Data Request to receive its assigned short network address.
- Finally, the coordinator sends a Transport Key packet (encrypted), which contains the Network Key.
- From this point forward, all communication is encrypted using this Network Key

```

21 293.0... 0xffff Beacon Request
22 293.2... 0x0000 Beacon, Src: 0x0000, EPID: NordicSemico_df:8d:12:5d:23
23 293.5... f4:ce:36:8f:c4:80:3d:c0 0x0000 Association Request, FFD
24 293.5... 0x0000 Ack
25 294.0... f4:ce:36:8f:c4:80:3d:c0 0x0000 Data Request
26 294.0... 0x0000 Ack
27 294.0... f4:ce:36:df:8d:12:5d:23 Association Response, PAN: 0xda88 Addr: 0x896b
28 294.0... 0x0000 Ack
29 294.0... 0x0000 0x896b Transport Key
30 294.0... 0x0000 Ack
31 294.1... 0x896b 0xffff Device Announcement, Nwk Addr: 0x896b, Ext Addr: NordicSemico_8f:c4:80:3d:c0

```

2.2 SpoofMe Badly:

Transport the Key

Why It Was Difficult

- The only major challenge I faced in simulating the endpoint connection was handling the Transport Key packet.
- All other Zigbee packets were relatively easy to implement.
 - The Transport Key packet is responsible for securely delivering the Network Key from the coordinator to the endpoint.
- This packet is encrypted using the Link Key, which is pre-shared between the coordinator and the endpoint.

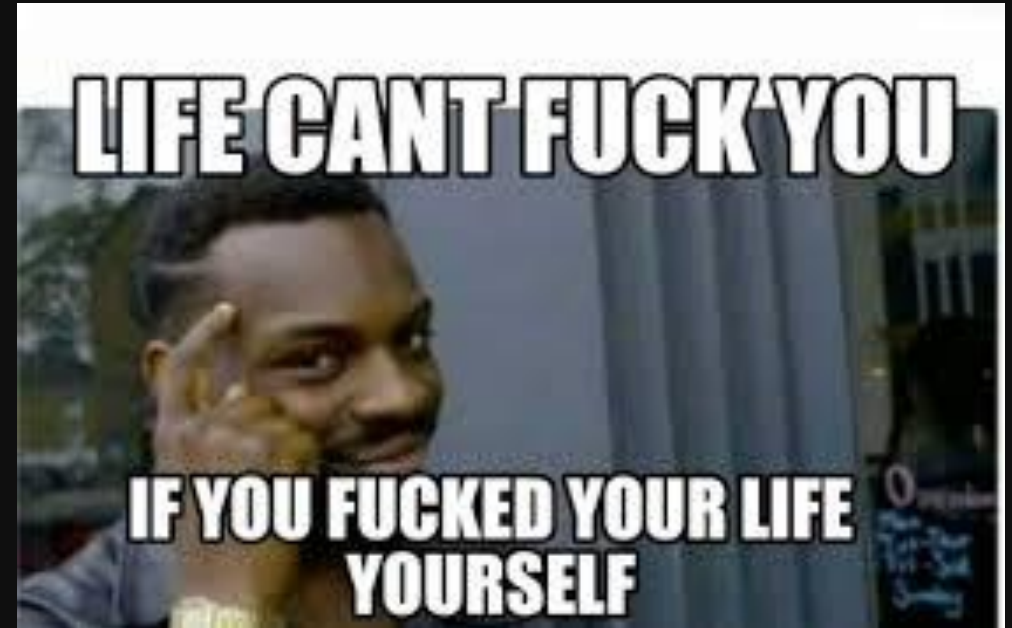
- Unlike encryption with the Network Key, the Link Key must first be hashed using a complex algorithm before it can be used for encryption.
- There's no straightforward Python implementation of this algorithm available.
- The only working version I found was the Wireshark implementation in C.
- I had to translate the algorithm from C to Python manually.
- Additionally, the encryption algorithm itself remains the same, but the input to the MAC (Message Authentication Code) is taken from different layers of the Zigbee packet to ensure message integrity.

```
def encrypt_transport(sec_header: ZigbeeSecurityHeader, plaintext_payload: bytes, aps1: bytes) -> bytes:
    nonce = craft_nonce(sec_header)
    cipher = AES.new(hash_key, AES.MODE_CCM, nonce, mac_len=4)
    cipher.update(bytes(aps1) + bytes(sec_header))
    ciphertext = cipher.encrypt(plaintext_payload)

    sec_header.data = ciphertext
    sec_header.mic = cipher.digest()
    sec_header.nwk_seclevel = 0
```

2.2 TurnMeOn TurnMeOff: The Wrong Way

- I was finally able to make the endpoint connect to my fake Zigbee coordinator.
- However, the customer increased the relay control security.
 - I started analyzing the system further using multiple interfaces.
 - I discovered that I could manipulate the relay by:
 - Sending specific Zigbee application layer commands.
- Receiving values from the endpoint to build the exploit logic.
 - The endpoint received my application layer commands successfully.
 - I was able to access different clusters.
 - But the endpoint never responded with any replies.
 - It seems like a routing issue is preventing the response.
- This means AGAIN I now need to implement basic Zigbee network layer functions to continue the assessment.



2.2 TurnMeOn TurnMeOff: The Wrong Way

- In any active Zigbee network, you'll notice many Link Status packets being sent periodically.
- These packets are used to report the quality of direct links to neighboring nodes.
 - Typically sent by routers and coordinators.
 - Contains a list of neighbors and link quality values.
- Helps update neighbor tables and detect changes in network topology
- Link Status packets include dynamic values like counter and must be encrypted using the network key.
 - Implementing this was not easy and stressful.
- Despite successfully implementing and sending Link Status packets,
 - the endpoint still didn't respond.
- This suggested that the issue was not solely related to link status, and further troubleshooting was needed.

```
▼ Command Frame: Link Status
  Command Identifier: Link Status (0x08)
  .1.. .... = Last Frame: True
  ..1. .... = First Frame: True
  ...0 0001 = Link Status Count: 1
  ▼ Link 1
    Address: 0x1e37
    .... .001 = Incoming Cost: 1
    .001 .... = Outgoing Cost: 1
```



2.2 TurnMeOn TurnMeOff:

The Final success

Pwned!

- After extensive debugging, I discovered the issue was at the Zigbee application layer.
- Each Zigbee application typically defines a "sink", which is the destination for data exchange — endpoints send or receive data through it.
- In our case, the endpoint required confirmation that the sink was alive. This was done by sending a specific application-layer packet to a designated endpoint and cluster containing custom data.
 - Once the endpoint received this sink announcement (a custom implementation by the vendor), it began to announce itself at the application layer as well.
- I had to analyze and exchange multiple values with the endpoint, crafting and sending various Zigbee application commands
- After fully implementing the necessary packets and logic, I was finally able to exploit the device and gain full control over the Zigbee endpoint, including manipulating the switch as intended.



Takeaways

- Always test with default keys first
- Begin with thorough online research existing work can save you time and guide your approach.
- Think creatively — experiment with modifying headers, values, and protocol behaviors.
 - Fuzzing can be useful in uncovering unexpected behaviors or vulnerabilities.
 - Don't rely on automated frameworks

Thanks for listening Questions



X: @haider_kabibo

